דוח מסכם- פרויקט רשתות תקשורת

מגישים:

גל דדון, ת.ז- 318557550
ניב שמש, ת.ז- 212879571
ליאורה רובינוב, ת.ז- 205659311

מרצה אחראית- ד״ר רויטל מרבל

קישורים: https://github.com/naybson/chat-project.git

חלק 1 - אריזת נתונים ולכידת מנות בעזרת wireshark

אריזה ולכידת המנות:

בשלב הראשון יצרנו קובץ CSV, שבו כל שורה מייצגת הודעות בשכבת היישום שנשלחות ב־TCP, כאילו זה נתונים שנעים ברשת אמיתית.
הרעיון היה לדמות תעבורה ברשת של הודעות שנשלחות בצורה סדרתית אחת אחרי השניה.
לאחר הכנת הקובץ, ע״י הרצת קוד, "שידרנו" את ההודעות מהרשימה אל ממשק loopback של המחשב. כך יכולנו לגרום למערכת לייצר תעבורה רשתית אמיתית.
בשלב הזה תיעדנו באמצעות Wireshark את החבילות העוברות, על ידי הפעלת Capture על ה־Loopback וצפינו בכל ההודעות כשהן "מתפרקות" לשכבות של תקשורת: החל מההודעה עצמה (Payload), דרך TCP, IP ועד שכבת ה־Data Link.
בכך תיעדנו את כל התעבורה בפועל לקובץ pcap על ידי אפליקציית wireshark.

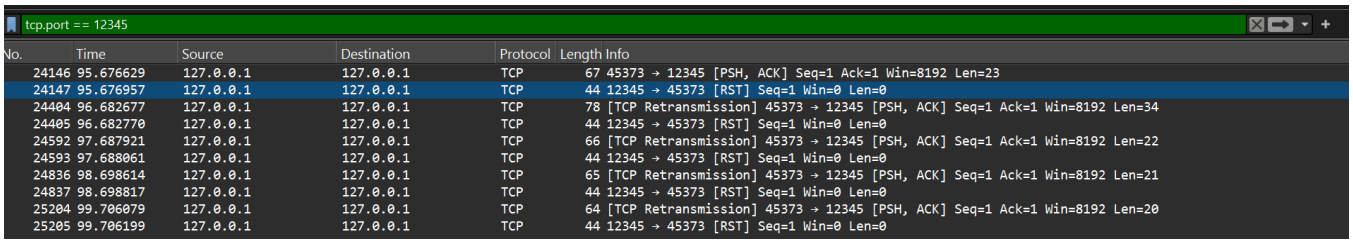
קובץ csv-

	msg_id	app_protocol	src_app	dst_app	message	timestamp
1	1	HTTP	client_browser	web_server	[CSV-1] GET /index.html	0.1
2	2	HTTP	client_browser	web_server	[CSV-2] POST /login username=alice	0.2
3	3	HTTP	client_browser	web_server	[CSV-3] GET /dashboard	0.3
4	4	HTTP	client_browser	web_server	[CSV-4] GET /settings	0.4
5	5	HTTP	client_browser	web_server	[CSV-5] POST /logout	0.5

*קובץ הcsv המקורי מצורף בתוצרים של חלק 1 בתיקיית גיטהאב של הפרויקט.

תיאור והסבר של תהליך הלכידה:

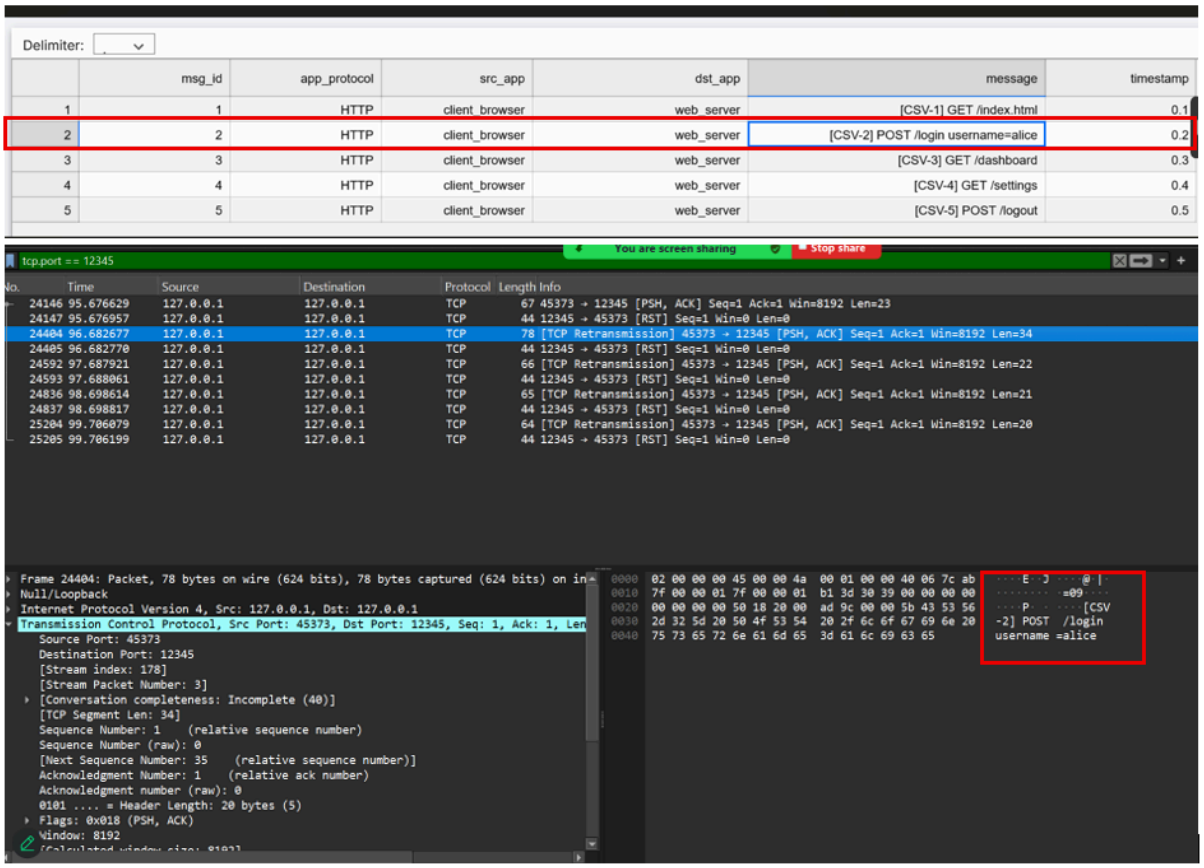
לאחר תיעוד התעבורה על ידי wireshark, השתמשנו בפילטר tcp.port == 12345 על מנת למצוא את ההודעות הדרושות.



tcp.port == 12345

No.	Time	Source	Destination	Protocol	Length	Info
24146	95.676629	127.0.0.1	127.0.0.1	TCP	67	45373 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=23
24147	95.676957	127.0.0.1	127.0.0.1	TCP	44	12345 → 45373 [RST] Seq=1 Win=0 Len=0
24404	96.682677	127.0.0.1	127.0.0.1	TCP	78	[TCP Retransmission] 45373 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=34
24405	96.682770	127.0.0.1	127.0.0.1	TCP	44	12345 → 45373 [RST] Seq=1 Win=0 Len=0
24592	97.687921	127.0.0.1	127.0.0.1	TCP	66	[TCP Retransmission] 45373 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=22
24593	97.688061	127.0.0.1	127.0.0.1	TCP	44	12345 → 45373 [RST] Seq=1 Win=0 Len=0
24836	98.698614	127.0.0.1	127.0.0.1	TCP	65	[TCP Retransmission] 45373 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=21
24837	98.698817	127.0.0.1	127.0.0.1	TCP	44	12345 → 45373 [RST] Seq=1 Win=0 Len=0
25204	99.706079	127.0.0.1	127.0.0.1	TCP	64	[TCP Retransmission] 45373 → 12345 [PSH, ACK] Seq=1 Ack=1 Win=8192 Len=20
25205	99.706199	127.0.0.1	127.0.0.1	TCP	44	12345 → 45373 [RST] Seq=1 Win=0 Len=0

על ידי כניסה לפרטי ההודעה, וידאנו כי אלו באמת ההודעות אותן שלחנו לרשת, על פי תוכן ההודעה והמאפיינים שלה.



Delimiter: ,

	msg_id	app_protocol	src_app	dst_app	message	timestamp
1	1	HTTP	client_browser	web_server	[CSV-1] GET /index.html	0.1
2	2	HTTP	client_browser	web_server	[CSV-2] POST /login username=alice	0.2
3	3	HTTP	client_browser	web_server	[CSV-3] GET /dashboard	0.3
4	4	HTTP	client_browser	web_server	[CSV-4] GET /settings	0.4
5	5	HTTP	client_browser	web_server	[CSV-5] POST /logout	0.5

לדוגמא, בשורה 2 של קובץ ה-CSV מופיעה ההודעה:

`CSV-2 POST username=alice`.

לאחר שליחת הודעות דרך מנגנון Raw TCP, ניתן לזהות ב-Wireshark חבילת TCP תואמת, שבה מופיע אותו תוכן בדיוק בשדה ה-payload של מקטע ה-TCP.

התאמה זו מאשרת כי הודעות שכבת היישום מה-CSV נארזו כ-payload בשכבת התעבורה והועברו כהלכה דרך שכבות הרשת.

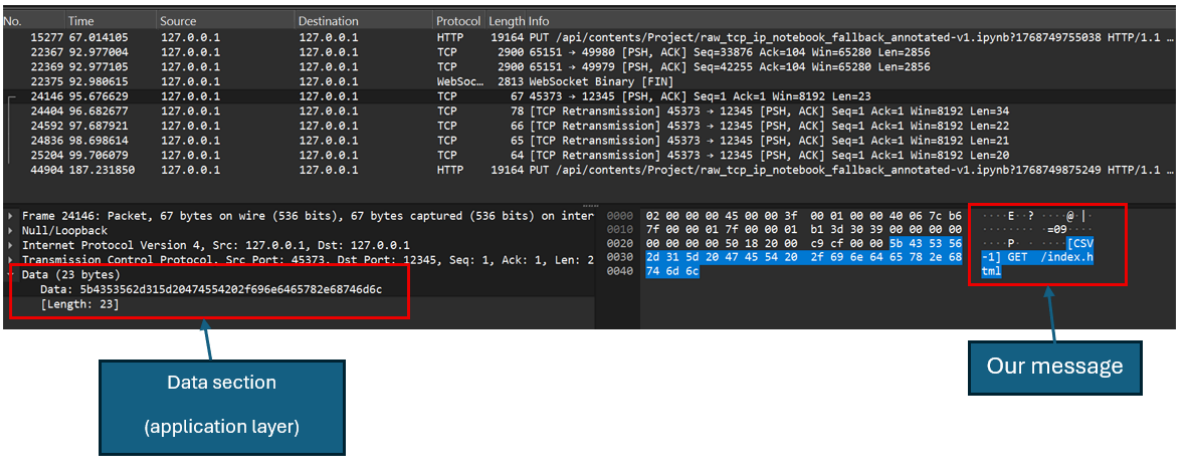
ניתוח חבילה ב Wireshark:

תהליך אריזת הודעות (Packet Encapsulation Process)

המטרה שלנו היא להעביר את הודעות שכבת היישום מתוך קובץ ה-CSV ולהפוך אותן למנות רשת באמצעות תהליך האריזה (Encapsulation).

א) שכבת היישום (יצירת ההודעה)

הודעות שכבת היישום נוצרות כטקסט רגיל (Plain Text) בשכבת היישום. במקרה זה, ההודעות מתועדות ונשמרות בקובץ CSV. בשלב זה, ההודעה מהווה נתון בלבד (Payload) ואינה כוללת מידע רשת כלשהו, כגון כתובות IP או מספרי פורטים.



ב) שכבת התעבורה (TCP Encapsulation)

בפרויקט שלנו, הודעות שכבת היישום נקראות מתוך קובץ ה-CSV ונשלחות דרך הרשת באמצעות Raw TCP.

כל הודעה מומרת לביטים ולאחר מכן נשלחות כ-payload של TCP.

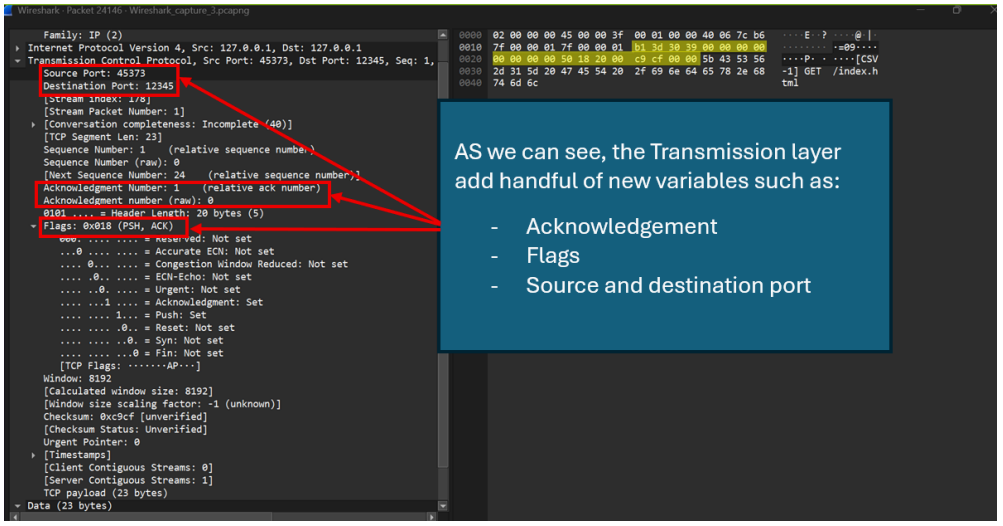
כאשר הודעה נשלחת, פרוטוקול TCP מוסיף לנתונים כותרת משלו (TCP Header).

כותרת זו כוללת מידע כגון:

- פורט מקור (פורט אקראי של הלקוח)
- פורט יעד (12345)
- דגלי (PSH and ACK)

התוצאה היא מקטע (TCP Segment):

(TCP Header | Application Data)



שכבת הרשת (IP Encapsulation)

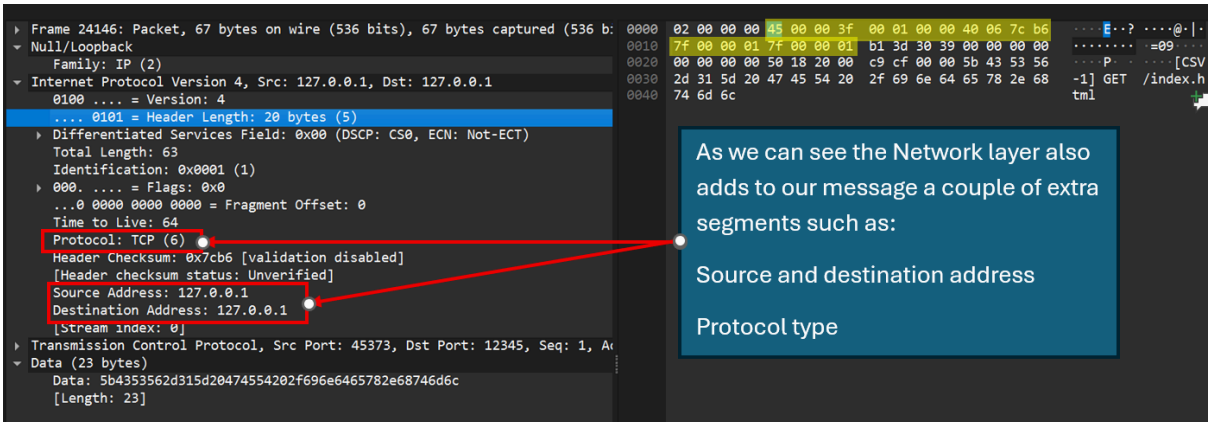
לאחר יצירת מקטע ה-TCP, הוא מועבר לשכבת ה-IP.
שכבת ה-IP מוסיפה כותרת IP למקטע ה-TCP.

כותרת זו כוללת:
- כתובת IP מקור
- כתובת IP יעד
- סוג הפרוטוקול (TCP)

בפרויקט שלנו, כתובת ה-IP של המקור ושל היעד היא 127.0.0.1, מכיוון שהלקוח והשרת רצים על אותו מחשב משתמשים בממשק ה-Loopback.

לאחר שלב זה, הנתונים הופכים למנת IP:

(IP Header | TCP Header | Application Message)



שכבת הקישור (Data Link Layer) – מיסגור Loopback

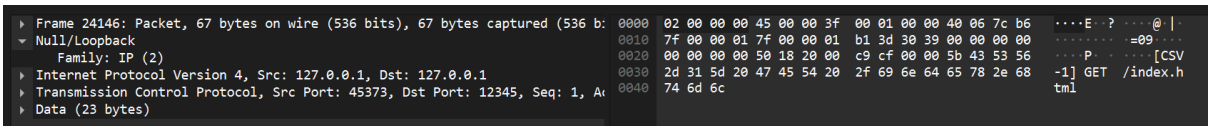
לאחר יצירת מנת ה-IP, היא נארזת במסגרת של שכבת הקישור (Data Link) לצורך שידור.

בפרויקט שלנו, התקשורת מתבצעת על אותו מחשב באמצעות כתובת ה-127.0.0.1 (Loopback).
לכן, במקום מסגרת Ethernet סטנדרטית, המנה מועברת באמצעות מסגרת Null (Loopback).

חשוב לציין כי ברשת פיזית אמיתית, שכבת הקישור הייתה כוללת כותרת Ethernet מלאה, אך בפרויקט זה נעשה שימוש בממשק Loopback ולכן מסגרת Ethernet אינה מופיעה.

פורמט האריזה הסופי הוא:

(Loopback Frame | IP Header | TCP Header | Application Message)



לסיכום

הודעות שכבת היישום נארזות תחילה על-ידי TCP, לאחר מכן על-ידי IP, ולבסוף במסגרת Loopback, לפני שהן נשלחות ליעדן.

ניתן בנוסף לעיין בקובץ לחידת wireshark מצורף בגיטהאב.

חלק 2 - כתיבת יישום רשת וניתוח תעבורה של אותו יישום

הפרויקט מממש מערכת צ'אט בזמן אמת בפרוטוקול TCP: שרת מרכזי שמקבל חיבורים ממספר לקוחות בו זמנית, משדר הודעות גלובליות לכולם, ומאפשר הודעות פרטיות (DM) בין שני משתמשים.
כל התקשורת ממומשת ישירות בעזרת Sockets בלבד, תוך שימוש בריבוי תהליכונים (Threads).

מבנה הקוד והקבצים

הפרויקט מחולק לקבצים הבאים:

- **server.py** - שרת TCP, ניהול לקוחות, Broadcast/DM, וריבוי תהליכונים.
- **client_logic.py** - לוגיקת לקוח: Handshake, קבלת הודעות ברקע, תור הודעות, ניהול היסטוריה ו-DM.
- **client_ui.py** - שכבת UI (ספריית Tkinter) שמממשת build_ui ומשתמשת בלוגיקה של client_logic.py.
- **main.py** - מייצר Launcher שמאפשר להרים שרת ולפתוח מספר לקוחות חדשים על ידי לחיצה.

נפרט על קובץ בנפרד ועל הפונקציות הקיימות בו:

תקשורת

- שימוש בפרוטוקול TCP (socket.AF_INET + socket.SOCK_STREAM).
- תקשורת דו-כיוונית: לקוחות שולחים הודעות לשרת, והשרת מחזיר הודעות חזרה ללקוחות.
- תמיכה בלפחות 5 לקוחות בו-זמנית: השרת יוצר Thread לכל חיבור לקוח ומטפל בלקוחות במקביל.

מבנה המערכת

- שרת: מאזין על HOST/PORT, מקבל חיבור, מקבל שם משתמש, ומתווך בין הלקוחות (Broadcast/DM).
- לקוח: מתחבר לשרת, שולח הודעות, ומקבל הודעות בזמן אמת.
- ממשק: במימוש הנוכחי יש ממשק גרפי בסיסי (Tkinter). הדרישה מאפשרת ממשק טקסטואלי - GUI הוא תוספת ולא חובה.

מגבלות טכניות וטיפול בשגיאות

- אין שימוש בספריות מוכנות לניהול שרתים/לקוחות - רק Sockets.
- טיפול בשגיאות: אימות שם משתמש, מניעת שמות שמורים/כפולים, טיפול בניתוק לא צפוי וניקוי משאבים.

פרוטוקול אפליקטיבי (Application Layer Protocol):

מאחר ש־TCP הוא פרוטוקול מבוסס זרם (Stream-Oriented) ואינו שומר גבולות בין הודעות אפליקטיביות, מספר הודעות עלולות להגיע יחד באותו מקטע או להתפצל בין מקטעים שונים.

לכן, בפרויקט זה כל הודעה אפליקטיבית מסתיימת בתו `/n` המשמש כסמן לסיום הודעה, והשרת והלקוח מפצלים את הזרם להודעות נפרדות באמצעות `splitlines()`.

סוגי הודעות עיקריים:

• Handshake: הלקוח שולח 'username>\n<' מיד אחרי connect. השרת מחזיר הודעת system או ERROR.

• הודעה גלובלית: הלקוח שולח 'text>\n<'. השרת משדר לכל הלקוחות: '[text> <username]\n'.

• DM: הלקוח שולח '/\n<text> <target> dm'. השרת שולח ליעד: '[text> <DM from <sender>'.

• פקודת יציאה: '/bye' - ניתוק יזום.

• רשימת משתמשים: השרת משדר לכולם '[system, ... :Online user1, user2]'.

מקביליות (Concurrency) וביצועים:

• בשרת: עבור כל חיבור חדש, נוצר Thread ייעודי שמריץ handle_client. מבנה clients מוגן ע"י Lock.
• בלקוח: Thread רקע קורא recv() ומכניס את המידע ל-Queue, וה-thread) UI הראשי) מעבד את ההודעות כל 100ms.

הוראות התקנה והרצה (אפשר להשתמש בזה לעשות תיקונים קטנים):

דרישות: Python 3.x (מומלץ 3.10+). אין תלות חיצונית מעבר ל-Tkinter (מגיע לרוב עם פייתון).

1) הורדה/שמירה של כל הקבצים באותה תיקייה: server.py, client_logic.py, client_ui.py, main.py.

2) הרצת שרת:

```
python server.py
```

3) הרצת לקוח (חלון GUI):

```
python client_ui.py
```

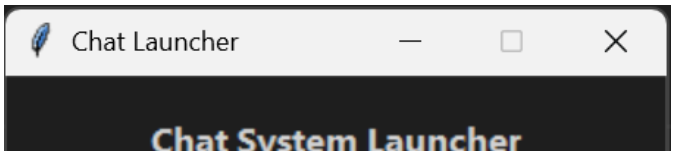
4) אופציה נוחה - Launcher שמרים שרת ופותח לקוחות:

```
python main.py
```

פירוט הקוד:

הפעלת השרת:

כאשר אנו פותחים את תוכנת הצ'אט יפתח החלון הבא:



חלון זה הוא חלון ההרצה של התוכנה שלנו, נועד כדי להתחיל את השרת ולחבר קליינטים לשרת בקלות.
במצב התחלתי נראה את כפתור Start Server אשר מתחיל את הרצת השרת, וכפתור Open New Client נעול ולא ניתן ללחיצה (כדי למנוע התחברות של לקוח לשרת שטרם נפתח).

בעת לחיצת הכפתור פונקצית Start server מורצת:

```
def start_server():
    # Starts the TCP server and accepts clients forever.
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    server.listen()

    print(f"Server listening on {HOST}:{PORT}")

    while True:
        sock, addr = server.accept()
        threading.Thread(
            target=handle_client,
            args=(sock, addr),
            daemon=True
        ).start()
```

הסבר פונקצית start server:

1) **יצירת socket של השרת**

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
```

נוצר אובייקט socket המשתמש בפרוטוקול `AF_INET` **IPV4** ובתקשורת `SOCK_STREAM` TCP.

2) **קישור השרת לכתובת ופורט:**

```
server.bind((HOST, PORT))
```

השרת נקשר לכתובת (HOST) ולפורט (PORT) שהוגדרו מראש, וכך יודע על איזה כתובת להאזין.
במיקרה שלנו port = 12345 וה-IP = 127.0.0.1

3) **התחלת האזנה לחיבורים**

```
server.listen()
```

השרת נכנס למצב האזנה ומוכן לקבל חיבורים מלקוחות.

4) **לולאה אינסופית לקבלת לקוחות**

```
    while True:
```

השרת רץ כל הזמן ומוכן לקבל לקוחות חדשים ללא הגבלה.

הלולאה האינסופית אנו מקבלים חיבורים חדשים ויוצרים Thread לכל לקוח.

א) קבלת חיבור מלקוח

```
sock, addr = server.accept()
```

השרת מקבל חיבור חדש:

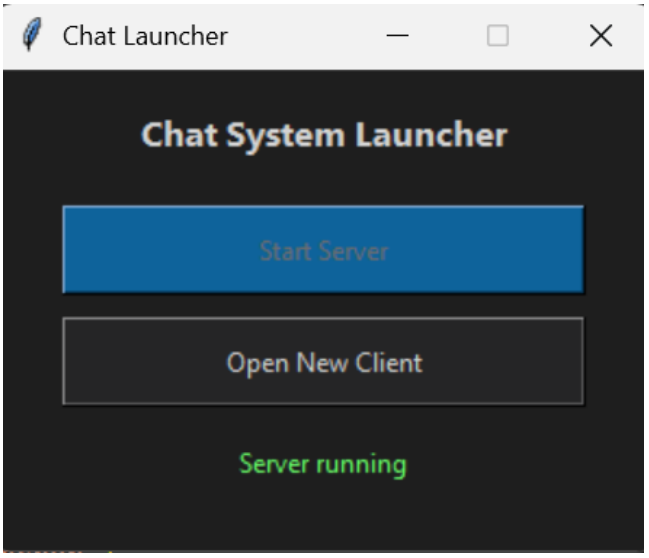
- socket – `sock` ייעודי לתקשורת עם הלקוח
- `addr` – כתובת ה־IP והפורט של הלקוח

ב) יצירת Thread לכל לקוח

```
threading.Thread(
    target=handle_client,
    args=(sock, addr),
    daemon=True
).start()
```

לכל לקוח נפתח Thread נפרד שמפעיל את הפונקציה `handle_client`.
כך השרת יכול לטפל במספר לקוחות בו־זמנית בלי לחסום חיבורים חדשים.
ה־Thread מוגדר כ־`daemon` כך שייסגר אוטומטית כאשר השרת נסגר.

לאחר פתיחת השרת חלון ההרצה מתעדכן באופן הבא:

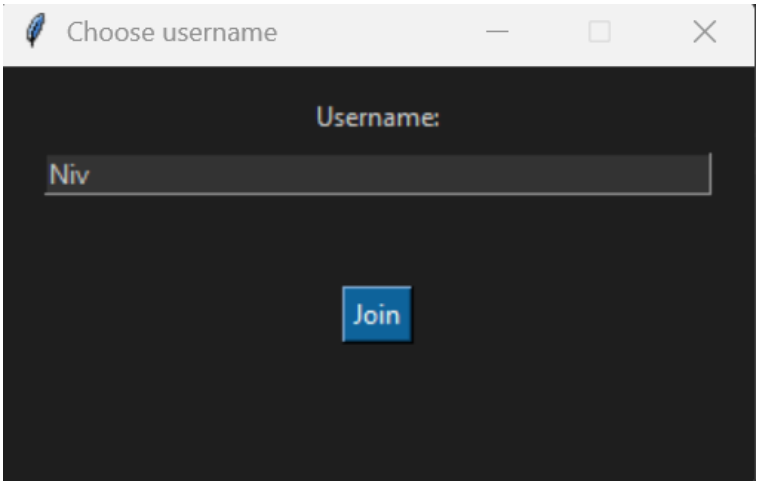


ניתן לראות שהשרת שלנו רץ! ולכן אפשר לנעול את כפתור התחלת השרת.
בנוסף נראה כפתור Open New Client שהיה נעול ניתן כעת ללחיצה.
בעת לחיצת הכפתור, נוכל לחבר לקוח חדש לשרת.

<u>התחברות לקוח</u>

1) פתיחת חלון בחירת שם משתמש (Client)

בעת הפעלת הלקוח, נפתח חלון ייעודי שבו המשתמש מתבקש להזין שם משתמש. שלב זה מתבצע במחלקה `UsernameDialog` בצד הלקוח.



תהליך ההתחברות מתחיל בצד הלקוח במחלקה `UsernameDialog`, האחראית על קבלת שם משתמש מהמשתמש וביצוע ה־handshake הראשוני מול השרת.

הלקוח אינו יוצר חיבור לשרת עד להזנת שם משתמש תקין. לאחר הזנת השם, נוצר חיבור TCP לשרת ונשלח שם המשתמש כשורה המסתיימת בתו `\n`, המסמן את סוף ההודעה.

הלקוח ממתין לתגובה ראשונית מהשרת המאשרת את ההתחברות או מחזירה שגיאה. רק לאחר קבלת אישור, מועבר ה־socket לשכבת הלוגיקה הראשית של הצ'אט.

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) # א
s.connect((HOST, PORT))                                # ב
s.send((name + "\n").encode())                         # ג
```

א) נוצר socket חדש המשתמש בפרוטוקול (`AF_INET` IPv4) ובתקשורת מבוססת TCP `SOCK_STREAM`.
Socket זה ישמש את הלקוח לכל התקשורת מול השרת.
ב) הלקוח יוזם חיבור TCP לשרת באמצעות כתובת ה־IP והפורט שהוגדרו מראש.
בשלב זה מתבצע ה־TCP handshake ברמת מערכת ההפעלה.
ג) הלקוח שולח לשרת את שם המשתמש שהוזן, כאשר התו `\n` משמש כסמן לסיום ההודעה.
השרת משתמש בתו זה כדי לקרוא את שם המשתמש כ"שורה מלאה" ולבצע אימות.

לאחר שהלקוח יוזם חיבור ושולח את שם המשתמש, השרת מקבל את החיבור באמצעות קריאה לפונקציה `accept`.

```
def start_server():

    # Starts the TCP server and accepts clients forever.
```

```
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.bind((HOST, PORT))
    print(f"Server listening on {HOST}:{PORT}")
    while True:
        sock, addr = server.accept()
        threading.Thread(
            target=handle_client,
            args=(sock, addr),
            daemon=True
        ).start()
```

עם קבלת החיבור, נוצר socket ייעודי עבור הלקוח, והטיפול בו מועבר ל־Thread נפרד, כך שהשרת יכול להמשיך ולהאזין לחיבורים נוספים במקביל.

בשלב הבא, השרת קורא את שם המשתמש שנשלח מהלקוח באמצעות פונקציית `recv_line`, הקוראת נתונים עד לתו `\n`, המסמן את סוף ההודעה.
שם המשתמש שנקלט עובר תהליך אימות הכולל בדיקות תקינות, כגון: תווים אסורים, שמות שמורים, ושמות שכבר נמצאים בשימוש.

```
def recv_line(sock):
    data = ""
    while not data.endswith("\n"):
        chunk = sock.recv(1).decode()
        if not chunk:
            return None
        data += chunk
    return data.strip()
```

אם אחת מהבדיקות נכשלת, השרת שולח ללקוח הודעת שגיאה וסוגר את החיבור.
במקרה של הצלחה, השרת רושם את המשתמש ברשימת המשתמשים המחוברים, שולח ללקוח הודעת אישור חיבור, ומעדכן את כלל הלקוחות ברשימת המשתמשים המקוונים.

רק לאחר השלמת שלבים אלו, המשתמש נחשב מחובר למערכת ויכול לשלוח ולקבל הודעות בצ'אט.

אימות משתמש והשלמת החיבור

לאחר קריאת שם המשתמש מהלקוח, השרת מבצע תהליך אימות שמטרתו להבטיח תקינות וזהות ייחודית של המשתמש במערכת.
האימות כולל בדיקות כגון פורמט השם, שמות שמורים ושמות שכבר מחוברים לשרת.

במקרה של כשל באחת הבדיקות, השרת שולח ללקוח הודעת שגיאה וסוגר את החיבור.
במקרה של הצלחה, המשתמש נרשם ברשימת המשתמשים המחוברים, מקבל הודעת אישור חיבור, והשרת מעדכן את כלל הלקוחות ברשימת המשתמשים המקוונים.

למשל אם משתמש יבחר שם שכבר מחובר במערכת נדפיס הודעת שגיאה.

```
with lock:
            if clean in clients:
                client_socket.send(
                    "[system] ERROR: Name already taken\n".encode())
                client_socket.close()
                return
```

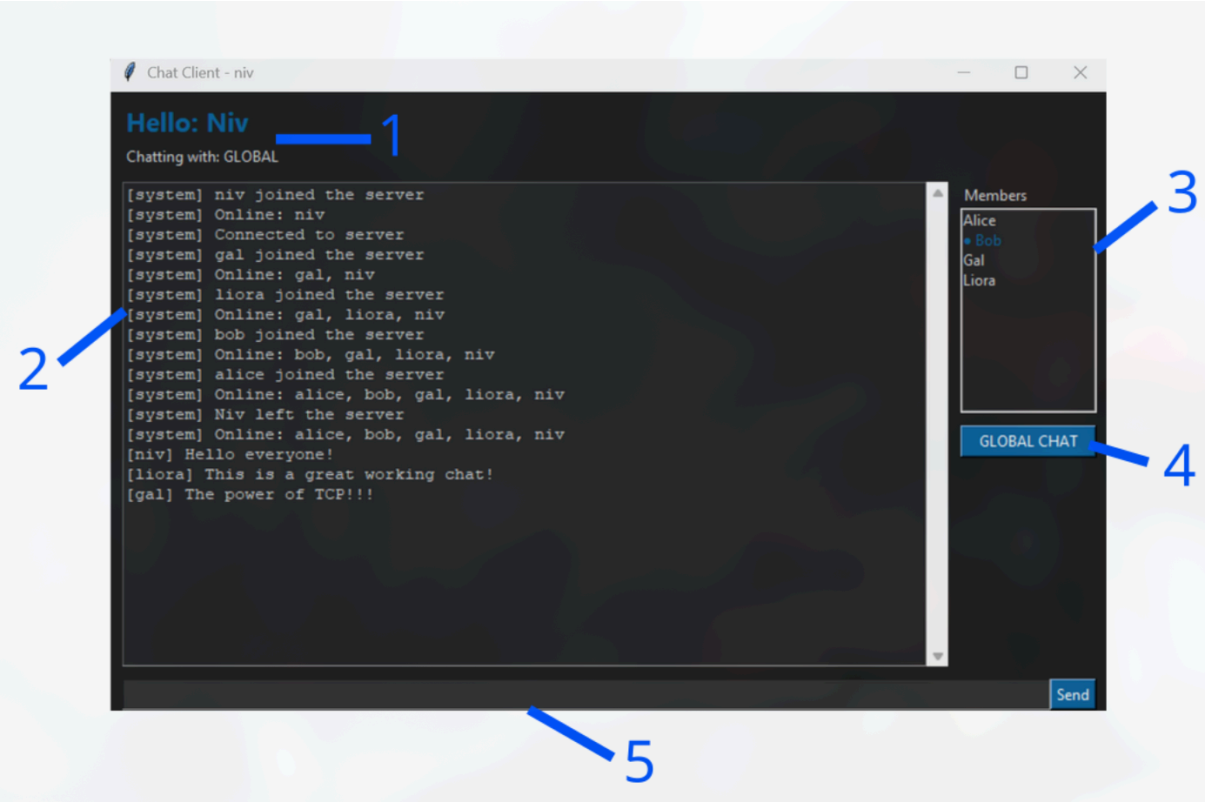
מאחר ותהליך האימות מתבצע בסביבת עבודה מרובת Threads, ייתכן שמספר לקוחות ינסו להתחבר בו־זמנית עם אותו שם משתמש.

לכן, בדיקת קיום השם והוספת המשתמש לרשימת הלקוחות מתבצעת בתוך מקטע נעול (`lock`), על מנת למנוע מצב שבו שני Threads מאשרים חיבור עם אותו שם עקב גישה מקבילית למבנה הנתונים.

שלב זה מסיים את תהליך ההתחברות ומעביר את הלקוח למצב עבודה רגיל.

מסך הצ'אט לאחר התחברות לשרת (Client UI Overview)

לאחר השלמת תהליך ההתחברות והאימות, מוצג למשתמש ממשק הצ'אט הראשי.
ממשק זה מאפשר צפייה בהודעות, בחירת סוג הצ'אט (גלובלי או פרטי), ושליחת הודעות למשתמשים אחרים.



להלן פירוט מרכיבי הממשק:

1) כותרת עליונה – מצב המשתמש והצ'אט הפעיל

בחלק העליון של החלון מוצג:

- שם המשתמש המחובר ("Hello: Niv")
- מצב הצ'אט הנוכחי ("Chatting with: GLOBAL")

אזור זה מתעדכן בהתאם להקשר הפעיל (צ'אט גלובלי או הודעה פרטית), ומאפשר למשתמש להבין בכל רגע לאן נשלחות ההודעות שהוא כותב.

2) אזור הצ'אט הראשי – הצגת הודעות

החלק המרכזי של המסך מציג את זרם ההודעות.
באזור זה מוצגות:

- הודעות מערכת (לדוגמה: משתמשים שהצטרפו או עזבו)
- הודעות צ'אט גלובליות
- הודעות פרטיות, כאשר המשתמש נמצא בצ'אט פרטי

הודעות מוצגות בסדר כרונולוגי ומופרדות לפי סוגן באמצעות תחיליות כגון `[username]`, `[system]`, או `[… DM from]`.

3) רשימת משתמשים מחוברים (Members)

בצד הימני של הממשק מוצגת רשימת המשתמשים המחוברים לשרת בזמן אמת.
הרשימה מתעדכנת אוטומטית בכל פעם שמשתמש מצטרף או עוזב את השרת.

בחירת משתמש מהרשימה מעבירה את המשתמש למצב צ'אט פרטי מול אותו משתמש.

4) כפתור GLOBAL CHAT

כפתור זה מאפשר חזרה מהירה לצ'אט הגלובלי מכל מצב אחר.
בעת לחיצה עליו, מוצגות הודעות הצ'אט הכללי והודעות חדשות נשלחות לכל המשתמשים המחוברים.

5) שדה הקלדת הודעה וכפתור שליחה

בתחתית החלון נמצא:

- שדה טקסט להקלדת הודעה
- כפתור "Send" (או לחיצה על Enter)

אזור זה משמש לשליחת הודעות לשרת, כאשר יעד ההודעה נקבע לפי מצב הצ'אט הנוכחי (גלובלי או פרטי).

שליחת הודעות

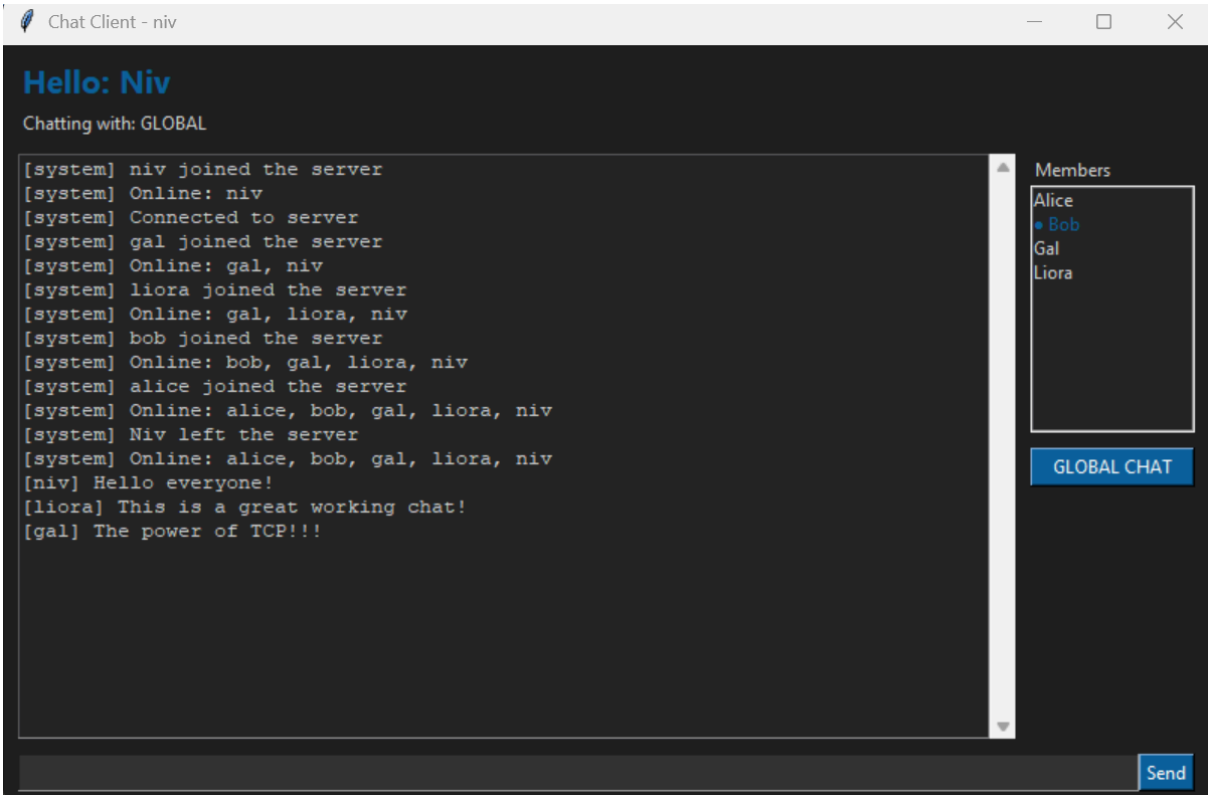
רק לאחר הבנת מבנה הממשק והקשרי העבודה שבו, ניתן להסביר את תהליך שליחת ההודעות עצמו:

- כיצד הלקוח שולח הודעה לשרת
- כיצד השרת מנתב אותה
- וכיצד היא מוצגת אצל היעד המתאים

שלב זה מתואר בסעיף הבא.

תהליך שליחת הודעות גלובליות (Global Messages)

לאחר שהמשתמש מחובר לשרת ונמצא במצב **GLOBAL CHAT**, הוא יכול לשלוח הודעה שתופץ לכל המשתמשים המחוברים במערכת.
התהליך מתבצע במספר שלבים ברורים, הכוללים את צד הלקוח, צד השרת, וצד הלקוחות המקבלים.



1) שליחת הודעה מצד הלקוח (Client → Server)

כאשר המשתמש מקליד הודעה ולוחץ על כפתור **Send** (או Enter), מופעלת פונקציית send () בצד הלקוח.

קטע קוד – שליחת הודעה גלובלית:

```
        msg = self.entry.get().strip()

        if not msg:
            return

        self.entry.delete(0, "end")

        if self.current_mode == "dm":

            self.sock.send(f"/dm {self.current_target}
{msg}\n".encode())

        else:

            self.sock.send((msg + "\n").encode())
```

הסבר:

- ההודעה נקראת משדה הקלט ונבדקת כדי למנוע שליחת הודעות ריקות
- ההודעה נשלחת לשרת דרך ה־socket
- התו \n מציין את סוף ההודעה ומאפשר לשרת לפצל הודעות שמגיעות ברצף

בשלב זה, הלקוח אינו יודע למי ההודעה תישלח – הוא רק שולח אותה לשרת.

2) קליטת ההודעה בצד השרת (Server Receive)

לאחר שהלקוח שלח את ההודעה, השרת קולט אותה בלולאת ההאזנה של הלקוח:

קטע קוד – קליטת הודעה:

```
        data = client_socket.recv(1024)
```

הסבר:

- הקריאה ל־`recv` חוסמת עד שמתקבל מידע מהלקוח
- אם מתקבל מידע ריק, המשמעות היא שהלקוח התנתק

לאחר מכן, השרת מפצל את המידע לשורות נפרדות:

```
        for msg in data.decode().splitlines():

        msg = msg.strip()
```

כך ניתן לטפל במספר הודעות שהגיעו יחד באותו מקטע TCP.

3) זיהוי הודעה גלובלית וניתוב (Routing)

בשלב זה, השרת בודק את סוג ההודעה.
אם ההודעה **אינה פקודה** (כגון `/dm`), היא נחשבת כהודעה גלובלית.

קטע קוד – הפצת הודעה גלובלית:

```
broadcast(f"[{username}] {msg}")
```

הסבר:

- השרת מוסיף את שם המשתמש להודעה
- ההודעה נשלחת לכל המשתמשים המחוברים
- השרת משמש כמתווך מרכזי ואינו שומר מצב של הצ'אט עצמו

4) שליחת ההודעה לכל הלקוחות (Server → Clients)

פונקציית `broadcast` שולחת את ההודעה לכל sockets של הלקוחות המחוברים:

```
def broadcast(message):

    # Send a message to all connected clients.

    # Errors are ignored so one broken client

    # does not crash the server.

    for sock in list(clients.values()):
```

```
        try:
            # we add \n to mark the end of the message
            sock.send((message + "\n").encode())
        except:
            pass
```

הסבר:

- ההודעה נשלחת לכל המשתמשים הפעילים
- גם השולח מקבל עותק של ההודעה
- \n משמש שוב כסמן סוף הודעה בצד הלקוח

5) קבלת ההודעה בצד הלקוחות והצגתה

בצד הלקוח, Thread רקע מאזין לשרת:

```
    def receive(self):
        while True:
            try:
                data = self.sock.recv(1024)
                if not data:
                    break
                self.queue.put(data.decode())
            except:
                break
```

הודעות מועברות ל־Queue בטוח, ולאחר מכן מטופלות ב־UI:

```
self.handle_message(msg.strip())
```

כאשר ההודעה מזוהה כהודעה גלובלית, היא:

- נשמרת בהיסטוריית הצ'אט הגלובלי
- מוצגת באזור הצ'אט הראשי

סיכום זרימת הודעה גלובלית

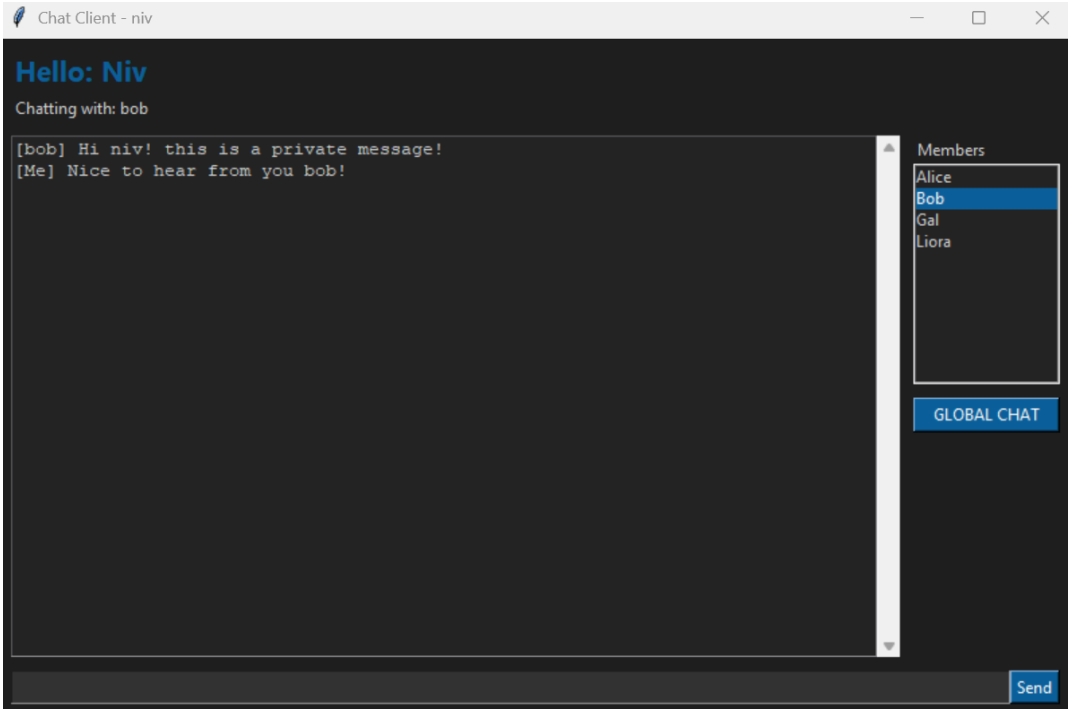
הודעה גלובלית זורמת במסלול:
Client → Server → Broadcast → All Clients

השרת משמש כמתווך מרכזי האחראי לניתוב והפצת ההודעות, בעוד שהלקוחות אחראים להצגתן בהתאם להקשר הפעיל.

תהליך שליחת הודעות פרטיות (Direct Messages – DM)

הודעות פרטיות מאפשרות תקשורת ישירה בין שני משתמשים, מבלי שההודעה תופץ לשאר המשתמשים המחוברים.

בשונה מהודעות גלובליות, הודעה פרטית מזוהה ומנותבת על־ידי השרת לפי פקודה ייעודית.



1) בחירת יעד והקלדת הודעה (Client)

בצד הלקוח, המשתמש בוחר משתמש מרשימת ה־**Members**, ועובר למצב צ'אט פרטי מולו.
מצב זה נשמר בלוגיקה של הלקוח (`"current_mode = "dm`) ו־`current_target`).

כאשר המשתמש מקליד הודעה ולוחץ Send, מופעלת פונקציית `send()`.

2) שליחת הודעת DM מהלקוח לשרת (Client → Server)

קטע קוד – שליחת הודעה פרטית:

```
if self.current_mode == "dm":

    self.sock.send(f"/dm {self.current_target}
```

```
            {msg}\n".encode())
```

הסבר:

- ההודעה נשלחת כטקסט רגיל דרך TCP הקידומת `/dm` משמשת כפקודת אפליקציה
- שם המשתמש של היעד והטקסט נשלחים באותה שורה
- `/n` מסמן את סוף ההודעה

חשוב לציין:
הודעות פרטיות אינן נשלחות בחיבור רשת נפרד. הפרטיות מנוהלת ברמת השרת, אשר מנתב את ההודעה ליעד יחיד בלבד.

3) קליטת הודעת DM בצד השרת

השרת קולט את ההודעה בלולאת `recv`, בדיוק כמו הודעה גלובלית:

```
        data = client_socket.recv(1024)
```

לאחר פיצול לשורות, השרת מזהה הודעה שמתחילה ב־`/dm`.

4) ניתוח וניתוב הודעת (DM Server Routing)

קטע קוד – טיפול ב־DM:

```
            if msg.startswith("/dm "):
                parts = msg.split(" ", 2)
                if len(parts) < 3:
                    send_to(
                        username,
                        "[system] Usage: /dm <user> <message>"
                    )
                    continue
                target, text = parts[1], parts[2]
```

הסבר:

- ההודעה מפוצלת לשלושה חלקים: פקודה, יעד, וטקסט
- השרת בודק:
  - שהיעד אינו השולח עצמו
  - שהיעד קיים ומחובר

במקרה של שגיאה, השרת שולח הודעת מערכת לשולח בלבד.

5) שליחת ההודעה ליעד ולשולח (Server → Clients)

אם כל הבדיקות עברו בהצלחה, השרת שולח:

ליעד:

```
send_to(target, f"[DM from {username}] {text}")
```

לשולח (Echo):

```
send_to(username, f"[DM to {target}] {text}")
```

הסבר:

- היעד מקבל הודעה פרטית עם ציון השולח
- השולח מקבל Echo לצורך הצגה מקומית בצ'אט
- ההודעה אינה נשלחת לאף לקוח אחר

6) קבלת הודעת DM בצד הלקוח

בצד הלקוח, הודעות DM מזוהות לפי תחילית:

- `[<DM from <user]` — הודעה נכנסת
- `[<DM to <user]` — הודעה יוצאת (Echo)

ההודעה:

- נשמרת בהיסטוריית ה־DM של אותו משתמש
- מוצגת רק בצ'אט הפרטי המתאים
- אם המשתמש אינו נמצא בצ'אט זה, מסומנת כהודעה שלא נקראה

סיכום זרימת הודעה פרטית

הודעה פרטית זורמת במסלול:

Client → Server → Target Client (+ Echo לשולח)

השרת אחראי לניתוב ולשמירת פרטיות ההודעה, בעוד שהלקוחות אחראים להצגתה בהקשר המתאים.

תהליך התנתקות משתמש (Disconnect)

במהלך העבודה התקינה של המערכת, ייתכן שמשתמש יתנתק מהשרת, בין אם כתוצאה מסגירת חלון הלקוח, תקלה ברשת, או שגיאה במהלך התקשורת.
במצב זה, בצד השרת מתבצעת קריאה ל־`recv`, אשר מחזירה מידע ריק או גורמת לשגיאה. מצב זה מזוהה כהתנתקות של הלקוח.

עם זיהוי ההתנתקות, השרת מסיר את המשתמש מרשימת המשתמשים המחוברים, סוגר את ה־socket הייעודי שלו, ושולח הודעת מערכת לשאר המשתמשים המחוברים על עזיבת המשתמש.
בנוסף, רשימת המשתמשים המקוונים (Members) מתעדכנת אצל כלל הלקוחות, כך שמצב המערכת נשאר עקבי ומעודכן.

תהליך זה מבטיח ניקוי משאבים תקין ושמירה על יציבות השרת גם במקרים של ניתוקים בלתי צפויים.

```
        while True:
            data = client_socket.recv(1024)
            if not data:
                break
```

שחרור נעילה בעת ניתוק:

```
    finally:
        # Cleanup on disconnect
        with lock:
            if username in clients:
                del clients[username]
        client_socket.close()
        if username:
            broadcast(f"[system] {username} left the server")
            broadcast_user_list()
```

מבני נתונים במערכת

במערכת הצ'אט נעשה שימוש במבנה נתונים מסוג **Dictionary (מילון)** לצורך ניהול המשתמשים המחוברים לשרת וה־sockets המשוייכים אליהם.

המילון מאפשר גישה מהירה ויעילה ללקוח לפי שם המשתמש שלו, דבר החיוני לניתוב הודעות, במיוחד בהודעות פרטיות.

מבנה הנתונים המרכזי – clients

בצד השרת מוגדר מבנה הנתונים הבא:

```
clients = {}
```

מבנה זה משמש כמילון שבו:

- **המפתח (key)** הוא שם המשתמש (username)
- **הערך (value)** הוא ה־socket המשמש לתקשורת עם אותו משתמש

כלומר, כל משתמש מחובר מיוצג על־ידי זוג:

`username → socket`

הוספת משתמש למבנה הנתונים

לאחר שהמשתמש עבר אימות בהצלחה, השרת מוסיף אותו למילון:

```
        with lock:

        clients[clean] = client_socket
```

הסבר:

- שם המשתמש משמש כמזהה ייחודי
- ה־socket נשמר לצורך שליחת הודעות עתידיות
- הפעולה מתבצעת בתוך `lock` כדי למנוע גישה מקבילית של Threads אחרים

שימוש במילון לשליחת הודעות

הודעות גלובליות

בשליחת הודעה גלובלית, השרת עובר על כל הערכים במילון (`sockets`) ושולח אליהם את ההודעה:

```
def broadcast(message):
```

```
    for sock in list(clients.values()):
        try:
            sock.send((message + "\n").encode())
        except:
            pass
```

כך ניתן להפיץ הודעה לכל המשתמשים המחוברים בקלות וביעילות.

הודעות פרטיות (DM)

בהודעות פרטיות, המילון מאפשר גישה ישירה ללקוח היעד לפי שם המשתמש שלו:

כאשר בפונקציה **send_to** מתבצעת שליפה מה־dictionary:

```
def send_to(username, message):
    sock = clients.get(username)
    if sock:
        sock.send((message + "\n").encode())
```

שימוש זה מאפשר ניתוב מדויק של הודעה למשתמש יחיד, ללא צורך במעבר על כלל המשתמשים.

הסרת משתמש ממבנה הנתונים בעת ניתוק

כאשר משתמש מתנתק מהשרת, השרת מסיר אותו מהמילון:

```
        with lock:
            if username in clients:
                del clients[username]
```

```
        client_socket.close()
```

כך מבטיחים שמצב המערכת משקף רק משתמשים מחוברים בפועל, ומונעים ניסיונות שליחה ל־socket שכבר נסגר.

למה נבחר Dictionary?

השימוש ב־Dictionary נבחר מכיוון שהוא:

- מאפשר גישה מהירה לפי שם משתמש O(1)
- מתאים לייצוג קשר חד־חד ערכי בין משתמש ל־socket
- מפשט את לוגיקת הניתוב של הודעות
- מאפשר ניהול מצב ברור ומרוכז של המערכת

ניתוח תעבורה (Wireshark) עד שכבת הרשת

מטרה: להראות את תעבורת ה-TCP ואת מסגרות ההודעות של האפליקציה.

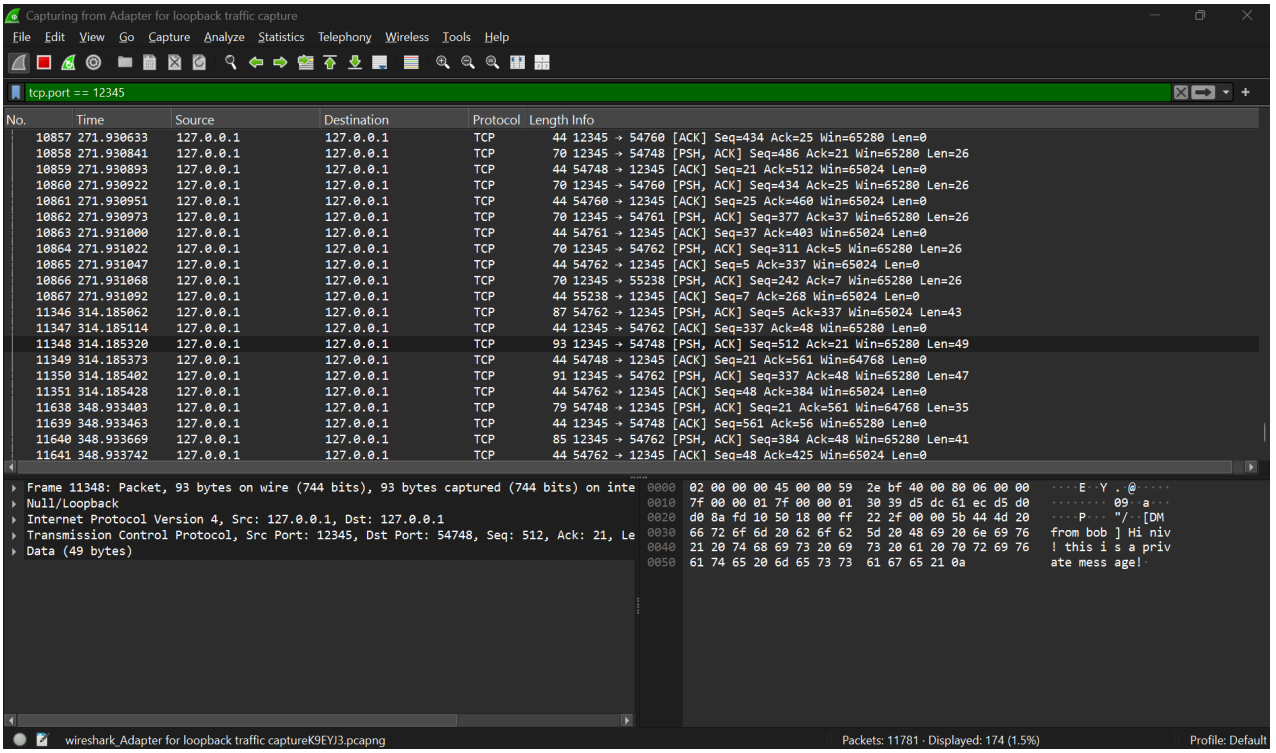
איך ללכוד תעבורה

1) לפתוח Wireshark ולבחור ממשק Loopback אם עובדים על אותו מחשב (127.0.0.1).

2) פילטר תצוגה מומלץ: tcp.port == 12345

3) להריץ את השרת ואז 2-3 לקוחות, לשלוח הודעות גלובליות ו-DM.

4) לחיצה ימנית על אחת החבילות > Follow > TCP Stream כדי לראות את הטקסט (payload) כזרם שורות.



מה צפוי לראות בשכבות

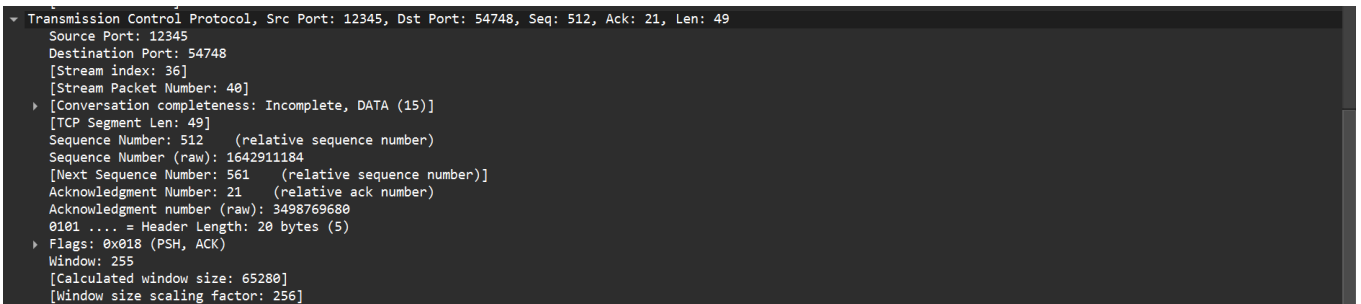
שכבת רשת (IP): מקור/יעד יהיו 127.0.0.1 כאשר השרת והלקוח באותו מחשב. ברשת אמיתית תראו כתובות LAN/WAN.

שכבת תעבורה (TCP):

- way handshake: SYN -> SYN/ACK -> ACK-3 בעת יצירת החיבור.
- Segmentation/Ack: הודעות האפליקציה יכולות להישבר למספר TCP segments, ויתקבלו ACK-ים בהתאם.
- סיום חיבור: FIN/ACK או RST במקרה של ניתוק/שגיאה.

שכבת אפליקציה (התוכן):

- בשדה TCP payload תראו מחרוזות ASCII/UTF-8 שמסתיימות ב-\n, למשל: 'alice\n' או 'alice]]' 'Hello\n'.
- ב-Follow TCP Stream ניתן להדגים את שלבי ה-handshake הלוגי: שם משתמש, הודעת system, ואז הודעות צ'אט.



```
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 89
    Identification: 0x2ebf (11967)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x0000 [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 127.0.0.1
    Destination Address: 127.0.0.1
    [Stream index: 0]
```

שימוש בבינה מלאכותית במהלך הפרויקט

במהלך העבודה על הפרויקט נעשה שימוש מבוקר בבינה מלאכותית ככלי עזר ללמידה, להכוונה ולשיפור תהליך הפיתוח, תוך שמירה על הבנה מלאה של הקוד והתוצרים הסופיים.

בחלק הראשון של הפרויקט, הבינה המלאכותית שימשה לצורך:

- סיוע ביצירת קובץ CSV תקין המכיל הודעות בשכבת האפליקציה, בהתאם לדרישות הפרויקט.
- הכוונה כללית כיצד לבצע לכידת תעבורה באמצעות תוכנת Wireshark, כולל שימוש בפילטרים מתאימים וניתוח בסיסי של חבילות שנלכדו.

הקובץ שנוצר והשלבים שבוצעו נבחנו, הותאמו ונבדקו על ידינו, וההסברים בדוח נכתבו לאחר הבנה של תהליך האריזה והלכידה.

פורמטים לדוגמא:

תכתוב לי קובץ CSV שידמה פקודות שכבות האפליקציה בפרוטוקול TCP,
העמודות בקבוץ צריכות להכיל את האלמנטים הבאים בסדר הזה: msg_id,
app_protocol, src_app, dst_app, message, timestamp

בטח — הנה קובץ CSV לדוגמה שמדמה פקודות "שכבת האפליקציה" שרצות מעל TCP (למשל אפליקציית צ'אט עם שרת שמתווך בין לקוחות).
אפשר להעתיק לקובץ בשם `tcp_app_commands.csv`.

Copy code csv

```csv
msg_id,app_protocol,src_app,dst_app,message,timestamp
1,CHATAPPv1,client_alice,server,"HELLO username=alice",2026-01-23T10:00:00.120+02:00
2,CHATAPPv1,server,client_alice,"WELCOME session_id=S-91A3",2026-01-23T10:00:00.165+02:00
3,CHATAPPv1,client_bob,server,"HELLO username=bob",2026-01-23T10:00:01.004+02:00
4,CHATAPPv1,server,client_bob,"WELCOME session_id=S-77C1",2026-01-23T10:00:01.049+02:00
5,CHATAPPv1,client_alice,server,"LIST_USERS",2026-01-23T10:00:02.310+02:00
6,CHATAPPv1,server,client_alice,"USERS alice,bob",2026-01-23T10:00:02.355+02:00
7,CHATAPPv1,client_alice,server,"OPEN_CHAT peer=bob",2026-01-23T10:00:03.020+02:00
8,CHATAPPv1,server,client_alice,"CHAT_OPENED chat_id=C-1007 peer=bob",2026-01-23T10:00:03.071+02:0
9,CHATAPPv1,server,client_bob,"INCOMING_CHAT chat_id=C-1007 peer=alice",2026-01-23T10:00:03.083+02
10,CHATAPPv1,client_bob,server,"CHAT_ACCEPT chat_id=C-1007",2026-01-23T10:00:03.410+02:00
11,CHATAPPv1,server,client_alice,"CHAT_ACCEPTED chat_id=C-1007 by=bob",2026-01-23T10:00:03.452+02:
12,CHATAPPv1,client_alice,server,"SEND chat_id=C-1007 text=""2026-01-23,""""הייי בוב!T10:00:04.010+02:
13,CHATAPPv1,server,client_bob,"DELIVER chat_id=C-1007 from=alice text=""2026-01-23,""""הייי בוב!T10:0
14,CHATAPPv1,client_bob,server,"SEND chat_id=C-1007 text=""2026-01-23,"""" 👋 הייי אליסT10:00:04.900+02
15,CHATAPPv1,server,client_alice,"DELIVER chat_id=C-1007 from=bob text=""2026-01-23,"""" 👋 הייי אליסT
16,CHATAPPv1,client_alice,server,"TYPING chat_id=C-1007 state=on",2026-01-23T10:00:05.220+02:00
```

+ Ask anything

ChatGPT can make mistakes. Check important info.

שאלה מצוינת — ובאמת קריטית לדוח שלך. בוא נהיה **חדים ולא מבלבלים**.

TL;DR (אם אין לך סבלנות)

אם אתה עובד על **loopback (localhost)** ורוצה לראות **הודעות אפליקציה מעל TCP**:

Copy code wireshark

```wireshark
tcp && ip.addr == 127.0.0.1
```

ואם אתה יודע את הפורט של השרת (מומלץ!):

Copy code wireshark

```wireshark
tcp.port == 12345
```

זה הפילטר הכי נקי, הכי נכון, והכי "מרצה אוהב".

עכשיו ההסבר כמו בן אדם 👇

1️⃣ **Loopback = localhost**

ב־Wireshark, תקשורת בין client ↔ server על אותו מחשב תעבור דרך:

בחלק השני של הפרויקט, הבינה המלאכותית שימשה ככלי לימודי לצורך:

- הבנת אופן השימוש בספריית `socket` בפייתון למימוש תקשורת TCP בין שרת ולקוחות.
- לימוד עקרונות עבודה עם ריבוי תהליכונים (Threads) לצורך טיפול במספר לקוחות בו־זמנית.
- קבלת רעיונות כלליים לעיצוב ממשק משתמש גרפי (GUI) נוח וברור באמצעות Tkinter.

הקוד הסופי נכתב, הותאם ונבדק על ידינו, תוך הבנה מלאה של מבנה המערכת, זרימת התקשורת והלוגיקה הממומשת.

השימוש בבינה מלאכותית נעשה ככלי תומך בלבד, ולא כתחליף להבנה, תכנון או מימוש עצמאי של הפרויקט.

פורמט לדוגמא:
תסביר לנו איך לממש תקשורת בסיסית מסוג לקוח–שרת בפרוטוקול TCP בשפת Python, באמצעות ספריית `socket`.

"תסביר לנו איך לממש תקשורת בסיסית מסוג לקוח–שרת בפרוטוקול TCP בשפת Python, באמצעות ספריית socket."

סגור. הנה הסבר **ברור, בסיסי ו"מרצה-פרנדלי"** על מימוש תקשורת לקוח–שרת ב-TCP בפייתון עם `socket`. בלי קפיצות, בלי קסמים.

מבוא קצר – מה אנחנו בכלל עושים?

תקשורת **לקוח–שרת (Client–Server)** היא מודל שבו:

- **שרת (Server)** מאזין לחיבורים נכנסים ומטפל בהם
- **לקוח (Client)** יוזם חיבור לשרת ושולח/מקבל נתונים

ב-TCP:

- החיבור **אמין** (אין איבוד מנות)
- הנתונים מגיעים **בסדר הנכון**
- יש חיבור מתמשך (לא "שלח וברח" כמו UDP)

ב-Python מממשים את זה בעזרת ספריית `socket`.

שלב 1️⃣ – יצירת שרת TCP

תפקידי השרת:

1. ליצור Socket
2. לקשור אותו ל-IP ופורט
3. להאזין לחיבורים
4. לקבל חיבור מלקוח
5. לקבל ולשלוח הודעות

שלבי העבודה (לוגית):

Copy code text

```
socket() → bind() → listen() → accept() → recv()/send()
```

באיזה ספריה כדאי לנו להשתמש כדי ליצור UI לפרוייקט שלנו? איך משתמשים בה?

" באיזה ספריה כדאי לנו להשתמש כדי ליצור UI לפרוייקט שלנו? איך משתמשים בה?"

שאלה מעולה — ופה אני הולך להיות **ישיר ואכזרי-לטובתך**:
לפרויקט כזה (צ'אט TCP, לימודי, עם דדליין ומרצה) — **Tkinter היא הבחירה הנכונה**. נקודה.

עכשיו נסביר גם *למה*, גם *איך*, וגם *למה לא לבחור דברים אחרים*.

באיזו ספרייה כדאי להשתמש ל-UI?

✅ Tkinter (הספרייה המובנית של Python)

למה Tkinter?

- ✔ **מובנית בפייתון** – אין התקנות, אין תקלות
- ✔ יציבה, פשוטה, מוכרת למרצים
- ✔ מושלמת לפרויקטי קורס
- ✔ עובדת טוב עם Threads (קריטי לצ'אט)
- ✔ קל להציג TextBox, כפתורים, חלונות

⚠️ היא לא "יפה" כמו React או Qt — אבל זה **לא רלוונטי לפרויקט אקדמי**.